

HuggingFace를 활용한
인공지능 웹 애플리케이션 개발

Chapter 8

배포와 운영

Uvicorn · Gunicorn · Docker · AWS · Nginx · CI/CD · Prometheus · Grafana

2025년판 개정증보

목 차

8장 서문	3
8.1 FastAPI 운영 환경: Uvicorn, Gunicorn	4
(1) FastAPI 개발 vs 운영 환경 차이	4
(2) Uvicorn 단독 실행의 한계와 Gunicorn 연동 이유	6
(3) 운영 환경 실행 명령어 및 설정 최적화	8
8.2 Hugging Face Spaces vs 자체 서버	11
(1) Hugging Face Spaces의 장점과 한계	11
(2) Gradio, Streamlit 앱 빠르게 올리기	13
(3) Spaces 사용법: app.py, requirements.txt, README.md	15
(4)(5) 자체 서버 비교 및 선택 기준	17
8.3 Docker로 API 감싸기	19
(1) Docker 기본 개념과 AI 서비스에서의 필요성	19
(2) Dockerfile: FastAPI + 모델 환경 이미지 생성	21
(3) 모델 파일 포함 여부에 따른 이미지 최적화	24
(4) Docker Compose와 Gunicorn 연동	26
8.4 AWS, Render 등 배포 실습	29
(1) AWSEC2 인스턴스에 Docker 이미지 배포	29
(2) SSH, 포트, 방화벽, Nginx 연동 실습	31
(3) Render, Railway.io 등 저비용 배포 플랫폼	34
(4)(5) 토큰/모델 캐싱 이슈 및 GitHub 자동 배포	36
8.5 API 모니터링과 속도 최적화	39
(1) 요청/응답 로그 남기기 (LoggingMiddleware)	39
(2) Uvicorn/Gunicorn 로그 설정	41
(3)(4) 응답 속도 측정 및 개선 전략	43
(5) Prometheus, Grafana, Sentry 모니터링 도구	45
8장 마무리 및 연습 문제	48

8장 배포와 운영

이 장에서는 지금까지 개발한 FastAPI 기반의 AI 웹 서비스를 실제 운영 환경에 배포하고 안정적으로 운영하는 전 과정을 다룹니다. 개발 서버에서만 돌아가는 코드를 넘어, 실제 사용자 트래픽을 받을 수 있는 프로덕션 수준의 서비스로 끌어올리는 것이 이 장의 핵심 목표입니다.

8.1절에서는 개발 환경과 운영 환경의 차이를 이해하고 Uvicorn/Gunicorn을 올바르게 설정합니다. 8.2절에서는 Hugging Face Spaces를 활용한 무료 배포와 자체 서버 운영을 비교합니다. 8.3절에서는 Docker로 애플리케이션을 컨테이너화하여 어디서나 동일하게 실행되는 환경을 구성합니다. 8.4절에서는 AWS EC2, Render 등 다양한 클라우드 플랫폼에 실제 배포를 실습합니다. 8.5절에서는 운영 중인 서비스의 성능을 모니터링하고 병목 지점을 최적화하는 방법을 다룹니다.

이 장에서 배울 내용

- Uvicorn + Gunicorn 조합으로 멀티 워커 운영 서버 구성
- Hugging Face Spaces에 Gradio/Streamlit AI 앱 무료 배포
- Dockerfile 작성 및 Docker 이미지 빌드/실행
- Docker Compose로 FastAPI + 모델 + Nginx 통합 배포
- AWS EC2, Render, Railway.io 실전 배포 실습
- LoggingMiddleware, Prometheus, Grafana로 실시간 모니터링
- GitHub Actions 기반 CI/CD 자동 배포 파이프라인 구성

이 장의 실습 환경

- Python 3.9 이상, pip, Docker Desktop (로컬)
- AWS 계정 (EC2 Free Tier 사용 가능)
- GitHub 계정 (CI/CD 파이프라인용)
- Hugging Face 계정 (Spaces 배포용)

- Render 또는 Railway.io 계정 (무료/저비용 대안)
- `pip install gunicorn prometheus-fastapi-instrumentator sentry-sdk loguru`

8.1 FastAPI 운영 환경: Uvicorn, Gunicorn

(1) FastAPI 개발 vs 운영 환경 차이

FastAPI 개발 서버(uvicorn --reload)는 코드 변경을 자동 감지하고 재시작하는 편의 기능이 있지만 단일 프로세스로만 동작하고 에러 처리가 느슨하여 실제 트래픽을 감당하기에 적합하지 않습니다.

운영 환경에서는 다중 워커(Multi-Worker) 프로세스, 구조화된 로깅, 안정적인 에러 처리, 보안 설정이 필수입니다.

구분	개발 환경 (Development)	운영 환경 (Production)
실행 명령	uvicorn main:app --reload	gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker
워커 수	1개 (싱글 프로세스)	CPU 코어 × 2 + 1 (멀티 프로세스)
자동 재시작	코드 변경 시 자동 재시작	크래시 발생 시에만 재시작
디버그 모드	debug=True, 상세 오류 노출	debug=False, 오류 은폐
로깅	콘솔 출력 (stdout)	파일 로테이션 + 구조화 JSON 로그
보안	CORS 전체 허용, HTTP	CORS 제한, HTTPS(Nginx), 인증
성능	낮음 (단일, 오버헤드 있음)	높음 (멀티워커, 최적화)

개발 환경과 운영 환경의 설정을 코드 레벨에서도 분리하는 것이 중요합니다. python-dotenv와 Pydantic Settings를 활용하면 환경변수 기반으로 설정을 손쉽게 관리할 수 있습니다.

```
# config.py - Pydantic Settings로 환경별 설정 관리
from pydantic_settings import BaseSettings
from functools import lru_cache
from typing import Literal

class Settings(BaseSettings):
    # 앱 기본 설정
    APP_NAME: str = "AI API Service"
```

```
APP_VERSION: str = "1.0.0"
ENVIRONMENT: Literal["development", "staging", "production"] = "development"
DEBUG: bool = False

# 서버 설정
HOST: str = "0.0.0.0"
PORT: int = 8000
WORKERS: int = 1

# 모델 설정
MODEL_ID: str = "runwayml/stable-diffusion-v1-5"
MODEL_CACHE_DIR: str = "./model_cache"
HF_TOKEN: str = ""

# 보안 설정
SECRET_KEY: str = "change-this-in-production"
ALLOWED_ORIGINS: list = ["*"]
API_KEY: str = ""

# 로깅 설정
LOG_LEVEL: str = "INFO"
LOG_FILE: str = "./logs/app.log"

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

    @property
    def is_production(self) -> bool:
        return self.ENVIRONMENT == "production"

    @property
    def recommended_workers(self) -> int:
        import os
        return (os.cpu_count() or 1) * 2 + 1
```

```
@lru_cache()
def get_settings() -> Settings:
    return Settings()

# .env.development 예시
# ENVIRONMENT=development
# DEBUG=True
# LOG_LEVEL=DEBUG
# WORKERS=1

# .env.production 예시
# ENVIRONMENT=production
# DEBUG=False
# LOG_LEVEL=INFO
# WORKERS=5
# SECRET_KEY=supersecretkeyhere
# HF_TOKEN=hf_xxxxxxxxxxxx
# ALLOWED_ORIGINS=["https://myapp.com","https://www.myapp.com"]
```

[코드 8-1] Pydantic Settings 기반 환경별 설정 분리 (config.py)

💡 python-dotenv와 Pydantic Settings를 함께 사용하면 .env 파일만 교체하여 개발/스테이징/운영 환경을 완전히 분리할 수 있습니다. 비밀 키, API 토큰 등 민감 정보는 절대 코드에 하드코딩하지 마세요.

(2) Uvicorn 단독 실행의 한계와 Gunicorn 연동 이유

Uvicorn은 ASGI(Asynchronous Server Gateway Interface) 서버로, 단일 이벤트 루프에서 비동기 요청을 처리합니다. 비동기 I/O 작업에서는 매우 효율적이지만, CPU를 집중적으로 사용하는 AI 모델 추론에서는 싱글 프로세스가 병목이 됩니다. Gunicorn은 이 문제를 해결해주는 프리포크(pre-fork) 워커 매니저입니다.

🔧 Uvicorn 단독의 한계

- ① 단일 프로세스 제약: GIL(Global Interpreter Lock)으로 인해 CPU 바운드 작업이 다른 요청을 블로킹
- ② 크래시 자동 복구 없음: 워커 프로세스가 비정상 종료되면 서비스 전체 중단
- ③ 부드러운 재시작(Graceful Reload) 미지원: 무중단 배포 불가
- ④ CPU 코어 활용 불가: 4코어 서버에서도 1코어만 사용

Gunicorn + UvicornWorker 조합의 장점

- ① 멀티 프로세스: 여러 Uvicorn 워커를 독립 프로세스로 실행 → 병렬 요청 처리
- ② 자동 워커 재시작: 워커 크래시 시 Gunicorn이 자동으로 새 워커 생성
- ③ 무중단 재시작: `gunicorn --reload` 또는 `SIGHUP` 시그널로 순차 재시작
- ④ 워커 수 동적 조정: 트래픽에 따라 `max_requests`, `timeout` 등 세밀한 조정 가능
- ⑤ 프로세스 격리: 한 워커의 메모리 누수가 다른 워커에 영향 없음

```
# — Uvicorn 단독 실행 (개발 전용) —
# 기본 개발 서버 - 단일 프로세스
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# — Gunicorn + UvicornWorker (운영 권장) —
# 기본 멀티워커 실행
gunicorn main:app ₪
  --worker-class uvicorn.workers.UvicornWorker ₪
  --workers 4 ₪
  --bind 0.0.0.0:8000
```

```
# —— Gunicorn 워커 수 자동 계산 ——  
# 일반 권장식: CPU 코어 수 × 2 + 1  
WORKERS=$(python3 -c "import os; print(os.cpu_count() * 2 + 1)")  
echo "권장 워커 수: $WORKERS"  
  
# AI 모델 서비스 특화: 메모리 집약적이므로 보수적으로 설정  
# GPU 메모리가 제한적이면 워커 수를 줄여야 함  
GPU_VRAM_GB=16  
MODEL_VRAM_GB=4  
MAX_WORKERS=$((($GPU_VRAM_GB / $MODEL_VRAM_GB))  
echo "VRAM 기반 최대 워커 수: $MAX_WORKERS"
```

```
# —— 완전한 운영 실행 명령어 ——  
gunicorn main:app ₩  
  --worker-class uvicorn.workers.UvicornWorker ₩  
  --workers 4 ₩  
  --bind 0.0.0.0:8000 ₩  
  --timeout 120 ₩  
  --keep-alive 5 ₩  
  --max-requests 1000 ₩  
  --max-requests-jitter 100 ₩  
  --graceful-timeout 30 ₩  
  --access-logfile ./logs/access.log ₩  
  --error-logfile ./logs/error.log ₩  
  --log-level info ₩  
  --pid ./gunicorn.pid
```

[코드 8-2] Gunicorn + UvicornWorker 실행 명령어 비교

(3) 운영 환경 실행 명령어 및 설정 최적화

Gunicorn 설정을 명령행 인자로 반복 입력하는 대신, gunicorn.conf.py 설정 파일로 관리하면 훨씬 체계적이고 버전 관리가 편리합니다.

```
# gunicorn.conf.py – Gunicorn 운영 설정 파일
import os
import multiprocessing

# — 서버 소켓 —
bind = "0.0.0.0:8000"
backlog = 2048 # 대기 중인 연결 수 (기본 2048)

# — 워커 설정 —
# AI 서비스: GPU 메모리 고려하여 워커 수 제한
# 일반 API: cpu_count * 2 + 1
cpu_count = multiprocessing.cpu_count()
workers = int(os.getenv("GUNICORN_WORKERS", min(cpu_count * 2 + 1, 4)))

worker_class = "uvicorn.workers.UvicornWorker"
worker_connections = 1000 # 워커당 최대 동시 연결 수 (async)

# 워커 수명 제한: 메모리 누수 방지 (1000요청 후 자동 재생성)
max_requests = 1000
max_requests_jitter = 100 # 동시 재시작 방지를 위한 랜덤 지연

# — 타임아웃 설정 —
# AI 추론은 시간이 오래 걸릴 수 있으므로 넉넉히 설정
timeout = 120 # 워커 타임아웃 (초). AI 추론 시간 고려
graceful_timeout = 30 # 부드러운 종료 대기 시간
keepalive = 5 # Keep-Alive 연결 유지 시간 (초)

# — 로깅 설정 —
```

```

import logging
errorlog = "./logs/gunicorn_error.log"
accesslog = "./logs/gunicorn_access.log"
loglevel = os.getenv("LOG_LEVEL", "info").lower()

# 액세스 로그 포맷: JSON 구조화 로그
access_log_format = ('{"time":"%(t)s","remote_ip":"%(h)s",'
                    '"method":"%(m)s","url":"%(U)s",'
                    '"status":"%(s)s","bytes":"%(b)s",'
                    '"duration_ms":"%(D)s","referer":"%(f)s"}')

# — 프로세스 관리 —
pidfile = "./gunicorn.pid"
daemon = False # True이면 백그라운드 데몬으로 실행

# — 보안 설정 —
forwarded_allow_ips = "*" # Nginx 리버스 프록시 사용 시
proxy_protocol = False

# — 워커 프로세스 초기화 —
def on_starting(server):
    import os
    os.makedirs("./logs", exist_ok=True)
    print(f"Gunicorn 시작: {workers} workers, timeout={timeout}s")

def worker_init(arbiter, worker):
    """각 워커 프로세스 시작 시 실행 (GPU 초기화 등)"""
    print(f"워커 {worker.pid} 시작")

def on_exit(server):
    print("Gunicorn 종료 완료")

# — 실행 명령어 (설정 파일 사용) —
# gunicorn main:app -c gunicorn.conf.py

```

```
# —— systemd 서비스로 등록 (리눅스 서버) ——  
# /etc/systemd/system/ai-api.service  
# [Unit]  
# Description=AI FastAPI Service  
# After=network.target  
#  
# [Service]  
# User=ubuntu  
# WorkingDirectory=/home/ubuntu/ai-api  
# ExecStart=/home/ubuntu/.venv/bin/gunicorn main:app -c gunicorn.conf.py  
# Restart=always  
# RestartSec=3  
# Environment="PATH=/home/ubuntu/.venv/bin"  
#  
# [Install]  
# WantedBy=multi-user.target  
  
# systemctl enable ai-api && systemctl start ai-api  
# systemctl status ai-api  
# journalctl -u ai-api -f # 실시간 로그 확인
```

[코드 8-3] gunicorn.conf.py 운영 설정 파일 + systemd 등록

8.2 Hugging Face Spaces vs 자체 서버

(1) Hugging Face Spaces의 장점과 한계

Hugging Face Spaces는 AI 모델 데모와 애플리케이션을 무료 또는 저비용으로 배포할 수 있는 호스팅 플랫폼입니다. Gradio 또는 Streamlit 앱을 저장소에 올리기만 하면 자동으로 빌드되고 공개 URL이 생성됩니다. 연구자, 학생, AI 애플리케이션 개발자들이 모델을 시연하고 공유하는 데 최적화되어 있습니다.

구분	무료 플랜 (Free)	Pro 플랜 (\$9/월)	Enterprise
CPU	2 vCPU, 16GB RAM	8 vCPU, 32GB RAM	커스텀
GPU	ZeroGPU (큐 대기)	A10G (24GB VRAM)	A100/H100
저장공간	50GB	100GB	무제한
동시 요청	제한적 (큐잉)	안정적	SLA 보장
개인 Space	공개만 가능	비공개 가능	비공개
커스텀 도메인	미지원	지원	지원
Sleep 정책	48h 미사용 시 슬립	없음	없음

비교 항목	Hugging Face Spaces	자체 서버 (AWS/GCP 등)
초기 설정 난이도	매우 쉬움 (git push 한 줄)	높음 (서버 설정, Docker, Nginx)
비용	무료 ~ 저비용	중간 ~ 높음 (GPU 인스턴스 비쌈)
커스터마이징	제한적 (Gradio/Streamlit 위주)	완전 자유
확장성	제한적 (수동 플랜 업그레이드)	Auto Scaling 가능
지속 실행	미사용 시 슬립 (무료)	24/7 실행 가능
보안/프라이버시	코드 공개 (무료)	완전 비공개 가능
모델 접근성	HF Hub 모델 바로 사용	모델 직접 다운로드/관리
권장 상황	학습용, 데모, 포트폴리오	실서비스, 기업 환경

(2) Gradio, Streamlit 앱 빠르게 올리기

Hugging Face Spaces에 앱을 배포하는 과정은 매우 간단합니다. Hugging Face Hub에 저장소를 만들고 app.py와 requirements.txt를 올리면 자동으로 배포됩니다.

```
# —— 방법 1: huggingface_hub 파이썬 패키지로 Space 생성 ——
```

```
pip install huggingface_hub
```

```
huggingface-cli login # HF 토큰으로 로그인
```

```
# Space 생성 (Python API)
```

```
from huggingface_hub import HfApi
```

```
api = HfApi()
```

```
# Gradio Space 생성
```

```
api.create_repo(
```

```
    repo_id="my-username/my-ai-demo",
```

```
    repo_type="space",
```

```
    space_sdk="gradio", # "gradio" 또는 "streamlit"
```

```
    private=False
```

```
)
```

```
# —— 방법 2: git clone → 파일 추가 → push ——
```

```
git clone https://huggingface.co/spaces/my-username/my-ai-demo
```

```
cd my-ai-demo
```

```
# 파일 추가 후
```

```
git add .
```

```
git commit -m "Add initial AI demo app"
```

```
git push
```

```
# —— 방법 3: huggingface_hub 업로드 API ——
```

```
from huggingface_hub import upload_file
```

```
upload_file(
```

```
    path_or_fileobj="./app.py",
```

```

path_in_repo="app.py",
repo_id="my-username/my-ai-demo",
repo_type="space"
)

# 배포된 Space URL: https://my-username-my-ai-demo.hf.space
# Hugging Face API URL: https://my-username-my-ai-demo.hf.space/api

```

[코드 8-4] Hugging Face Space 생성 및 배포 방법 3가지

```

# app.py – Gradio 기반 이미지 설명 AI 앱 (Space용)
import gradio as gr
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from PIL import Image

# ——— 모델 로드 (Space 빌드 시 자동 실행) ———
model_id = "vikhyatk/moondream2"

tokenizer = AutoTokenizer.from_pretrained(model_id, revision="2024-08-26")
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    trust_remote_code=True,
    revision="2024-08-26",
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32
)
model.eval()

# ——— 추론 함수 ———
def describe_image(image: Image.Image, question: str) -> str:
    if image is None:
        return "이미지를 업로드해주세요."
    if not question.strip():
        question = "이 이미지에 대해 설명해주세요."
    enc = model.encode_image(image)
    return model.answer_question(enc, question, tokenizer)

```

```
# —— Gradio UI

with gr.Blocks(title="AI 이미지 설명기") as demo:
    gr.Markdown("## 🖼️ AI 이미지 설명기")
    gr.Markdown("이미지를 업로드하고 질문을 입력하면 AI가 답변합니다.")
    with gr.Row():
        with gr.Column():
            img_input = gr.Image(type="pil", label="이미지 업로드")
            q_input = gr.Textbox(label="질문", value="이 이미지를 설명해주세요.")
            btn = gr.Button("분석", variant="primary")
        with gr.Column():
            output = gr.Textbox(label="AI 답변", lines=6)
    btn.click(describe_image, inputs=[img_input, q_input], outputs=output)
    gr.Examples(
        examples=[["./sample1.jpg", "이미지의 주요 내용은?"],
                  ["./sample2.jpg", "분위기를 설명해주세요."]],
        inputs=[img_input, q_input]
    )

demo.launch()
```

[코드 8-5] Gradio 기반 이미지 설명 앱 (app.py)

(3) Spaces 사용법: app.py, requirements.txt, README.md

Hugging Face Space는 다음 세 가지 파일이 핵심입니다. app.py(또는 app.ipynb)가 실행 진입점이고, requirements.txt가 의존성을 정의하며, README.md가 Space의 메타데이터를 포함합니다.

📁 Hugging Face Space 저장소 구조

my-ai-demo/

—— app.py	— 앱 진입점 (필수). Gradio 또는 Streamlit 앱
—— requirements.txt	— Python 패키지 의존성 (필수)
—— README.md	— Space 메타데이터 및 설명 (필수)
—— .gitattributes	— Git LFS 설정 (대용량 파일)
—— sample1.jpg	— 예시 이미지 (선택)
—— utils.py	— 보조 유틸리티 모듈 (선택)

```
# requirements.txt - Python 패키지 의존성
# 버전을 명시하면 빌드 재현성이 보장됨 (권장)

gradio==4.36.1
transformers==4.40.1
torch==2.2.2
torchvision==0.17.2
accelerate==0.29.3
Pillow==10.3.0

# 선택적 패키지
# diffusers==0.27.2    # 이미지 생성 필요 시
# opencv-python-headless # 이미지 처리 필요 시
# sentencepiece        # 일부 토큰라이저 필요 시

#

# README.md - Space 메타데이터 헤더
---
title: AI 이미지 설명기
```

```

emoji: 🤖
colorFrom: blue
colorTo: purple
sdk: gradio
sdk_version: 4.36.1
app_file: app.py
pinned: false
license: apache-2.0
short_description: 이미지를 업로드하면 AI가 설명해드립니다
models:
  - vikhyatk/moondream2
---

# AI 이미지 설명기

이미지를 업로드하고 질문을 입력하면 AI가 자세히 답변합니다.

## 사용 방법
1. 이미지 파일을 업로드하세요 (JPG, PNG 지원)
2. 질문을 입력하세요 (예: "이 이미지에 무엇이 보이나요?")
3. [분석] 버튼을 클릭하세요

## 모델 정보
- 기반 모델: [moondream2](https://huggingface.co/vikhyatk/moondream2)
- 파라미터: 1.8B (경량 멀티모달 모델)

```

[코드 8-6] requirements.txt 및 README.md 작성 가이드

(4)(5) 자체 서버 배포 대비 비교 및 선택 기준

프로젝트의 목적, 규모, 예산에 따라 최적의 배포 전략이 달라집니다. 학습/시연용과 실서비스 사이의 선택 기준을 명확히 이해하는 것이 중요합니다.

📁 배포 플랫폼 선택 기준 체크리스트

Hugging Face Spaces를 선택해야 할 때:

- 개인 포트폴리오, 연구 데모, 강의/세미나 시연 목적
- 예산이 거의 없는 학생/연구자
- HF Hub 모델을 그대로 사용하는 경우
- 빠른 프로토타입 배포 (30분 내 완성)

자체 서버(AWS/GCP/Render 등)를 선택해야 할 때:

- 실제 사용자 대상 프로덕션 서비스
- 지속적인 API 응답이 필요한 경우 (슬립 불가)
- 사용자 데이터 프라이버시/보안이 중요한 경우
- 커스텀 도메인, HTTPS, 인증 등 세밀한 제어 필요
- 모델을 직접 파인튜닝하여 운영하는 경우

8.3 Docker로 API 감싸기

(1) Docker 기본 개념과 AI 서비스에서의 필요성

Docker는 애플리케이션과 그 실행 환경을 하나의 컨테이너(Container)로 패키징하는 플랫폼입니다. "내 노트북에서는 되는데 서버에서는 안 돼요" 문제를 완전히 해결합니다. AI 서비스에서 Docker가 특히 중요한 이유는 Python 패키지 버전, CUDA 버전, 시스템 라이브러리 등 의존성이 복잡하기 때문입니다.

Docker 핵심 개념	설명	비유
Image (이미지)	컨테이너의 설계도/스냅샷. 읽기 전용 레이어 구조	요리 레시피
Container (컨테이너)	이미지를 실행한 인스턴스. 격리된 실행 환경	레시피로 만든 요리
Dockerfile	이미지를 빌드하는 스크립트 파일	레시피 문서
Registry (레지스트리)	이미지를 저장/공유하는 저장소. Docker Hub, ECR 등	레시피 공유 사이트
Docker Compose	여러 컨테이너를 정의하고 함께 실행하는 도구	폴코스 요리 메뉴
Volume (볼륨)	컨테이너 외부에 데이터를 영구 저장하는 방법	냉장고 (데이터 보관)
Network (네트워크)	컨테이너 간 통신을 위한 가상 네트워크	주방 내 통신망

```
# — Docker 설치 확인 —
docker --version
# 출력: Docker version 25.0.3, build 4debf41

docker compose version
# 출력: Docker Compose version v2.24.6

# — Docker 기본 명령어 정리 —
# 이미지 빌드
```

```
docker build -t my-ai-api:latest .

# 컨테이너 실행
docker run -d --name ai-api -p 8000:8000 my-ai-api:latest

# GPU 사용 컨테이너 실행
docker run -d --name ai-api --gpus all -p 8000:8000 my-ai-api:latest

# 실행 중인 컨테이너 목록
docker ps

# 컨테이너 로그 확인
docker logs -f ai-api

# 컨테이너 내부 접속
docker exec -it ai-api /bin/bash

# 컨테이너 중지 및 삭제
docker stop ai-api && docker rm ai-api

# 이미지 삭제
docker rmi my-ai-api:latest

# Docker Hub에 이미지 푸시
docker tag my-ai-api:latest myusername/my-ai-api:latest
docker push myusername/my-ai-api:latest
```

[코드 8-7] Docker 기본 명령어 정리

(2) Dockerfile로 FastAPI + 모델 환경 이미지 생성

AI 서비스를 위한 Dockerfile 작성에는 여러 주요 고려사항이 있습니다. CUDA 버전 선택, 패키지 레이어 캐싱 최적화, 불필요한 파일 제외, 보안 설정 등을 체계적으로 작성해야 합니다.

```
# Dockerfile – FastAPI AI 서비스 이미지
#
# ——— 베이스 이미지 선택 ———
# CUDA 필요 없는 경우 (CPU 전용)
# FROM python:3.11-slim
#
# CUDA 12.1 + cuDNN 8 (GPU 서버 배포 시)
FROM nvidia/cuda:12.1.0-cudnn8-runtime-ubuntu22.04
#
# ——— 환경 변수 설정 ———
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONFAULTHANDLER=1 \
    PIP_NO_CACHE_DIR=off \
    PIP_DISABLE_PIP_VERSION_CHECK=on \
    DEBIAN_FRONTEND=noninteractive
#
# ——— 시스템 패키지 설치 ———
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.11 python3.11-dev python3-pip \
    build-essential curl git wget \
    libgl1 libgl2.0-0 libsm6 libxext6 libxrender-dev \
    libgomp1 \
    && rm -rf /var/lib/apt/lists/*
#
RUN ln -sf /usr/bin/python3.11 /usr/bin/python3 \
    && ln -sf /usr/bin/python3 /usr/bin/python
#
# ——— 작업 디렉토리 생성 ———
WORKDIR /app
```

```
# —— 보안: 비루트 사용자 생성 ——
```

```
RUN groupadd --gid 1001 appuser ₩  
&& useradd --uid 1001 --gid 1001 --create-home appuser
```

```
# —— Python 의존성 설치 ——
```

```
# requirements.txt를 먼저 복사하여 레이어 캐시 활용
```

```
COPY requirements.txt .
```

```
RUN pip install --upgrade pip ₩
```

```
&& pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121 ₩
```

```
&& pip install -r requirements.txt ₩
```

```
&& pip install gunicorn
```

```
# —— 애플리케이션 코드 복사 ——
```

```
# .dockerignore로 불필요한 파일 제외 후 복사
```

```
COPY --chown=appuser:appuser . .
```

```
# —— 로그/캐시 디렉토리 생성 ——
```

```
RUN mkdir -p /app/logs /app/outputs /app/model_cache ₩
```

```
&& chown -R appuser:appuser /app
```

```
# —— 비루트 사용자로 전환 ——
```

```
USER appuser
```

```
# —— 포트 노출 ——
```

```
EXPOSE 8000
```

```
# —— 헬스체크 설정 ——
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 ₩
```

```
CMD curl -f http://localhost:8000/health || exit 1
```

```
# —— 실행 명령 ——
```

```
# 개발: uvicorn (간단하게)
```

```
# CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# 운영: gunicorn + uvicorn worker
CMD ["gunicorn", "main:app",
     "--worker-class", "uvicorn.workers.UvicornWorker",
     "--workers", "2",
     "--bind", "0.0.0.0:8000",
     "--timeout", "120",
     "--access-logfile", "/app/logs/access.log",
     "--error-logfile", "/app/logs/error.log"]
```

[코드 8-8] 운영 수준 Dockerfile (CUDA + 비루트 + 헬스체크)

```
# .dockerignore – 이미지에 포함하지 않을 파일 목록
__pycache__/_
*.py[cod]
*$py.class
*.so
.git/
.gitignore
.env
.env.*
*.log
logs/
outputs/

# 모델 캐시 제외 (별도 볼륨으로 관리)
model_cache/
*.safetensors
*.bin
*.ckpt
*.pth

# 개발 도구 제외
.venv/
venv/
node_modules/
*.egg-info/
dist/
build/
```

```
# 테스트 파일 제외
tests/
pytest.ini
*.test.py

# ——— 이미지 빌드 및 실행 ———
# 이미지 빌드
docker build -t ai-fastapi:latest .

# 빌드 시간 단축: BuildKit 사용
DOCKER_BUILDKIT=1 docker build -t ai-fastapi:latest .

# 빌드 결과 확인
docker images ai-fastapi

# CPU 컨테이너 실행
docker run -d \
  --name ai-api \
  -p 8000:8000 \
  -e HF_TOKEN=hf_xxxx \
  -v $(pwd)/model_cache:/app/model_cache \
  -v $(pwd)/logs:/app/logs \
  ai-fastapi:latest

# GPU 컨테이너 실행 (nvidia-container-toolkit 필요)
docker run -d \
  --name ai-api-gpu \
  --gpus all \
  -p 8000:8000 \
  -e HF_TOKEN=hf_xxxx \
  -v $(pwd)/model_cache:/app/model_cache \
  ai-fastapi:latest

# 실행 상태 확인
docker logs -f ai-api
docker inspect ai-api
```

[코드 8-9] .dockerignore 작성 및 빌드/실행 명령어

(3) 모델 파일 포함 여부에 따른 이미지 최적화 전략

AI 서비스 Docker 이미지의 가장 큰 문제는 모델 파일의 크기입니다. Stable Diffusion은 4~10GB, LLaVA-7B는 14GB를 초과합니다. 이 모델 파일을 Docker 이미지에 포함할지 외부에서 로드할지는 배포 전략에 큰 영향을 미칩니다.

전략	모델 포함 여부	이미지 크기	컨테이너 시작 시간	권장 상황
전략 A: 이미지 내 포함	O (이미지에 번들)	매우 큼 (10~20GB)	즉시 (다운로드 불필요)	오프라인 환경, 엣지 배포
전략 B: HF Hub 다운로드	X (런타임 다운로드)	작음 (~2GB)	느림 (첫 기동 시 수 GB 다운)	클라우드, HF Hub 접근
전략 C: Docker 볼륨 마운트	X (외부 볼륨)	작음 (~2GB)	빠름 (로컬 파일)	프로덕션 서버, 모델 공유
전략 D: S3/스토리지 다운로드	X (S3에서 다운)	작음 (~2GB)	빠름 (내부 네트워크)	AWS 배포, 프라이빗 모델

—— 전략 C: Docker 볼륨으로 모델 마운트 (권장 - 프로덕션) ——

1. 호스트 서버에 모델 미리 다운로드

```
python3 -c "
from huggingface_hub import snapshot_download
snapshot_download(
    repo_id='runwayml/stable-diffusion-v1-5',
    local_dir='/data/models/stable-diffusion-v1-5',
    token='hf_xxxxx'
)"
```

2. 볼륨 마운트로 컨테이너 실행

```
docker run -d ₩
--name ai-api ₩
--gpus all ₩
-p 8000:8000 ₩
-v /data/models:/app/model_cache:ro ₩
```

```

-e MODEL_CACHE_DIR=/app/model_cache ₩
ai-fastapi:latest

# ——— 전략 B: 런타임 다운로드 + 볼륨 캐싱 ———
# 첫 실행 시 다운로드, 이후 캐시 사용
docker run -d ₩
  --name ai-api ₩
  -p 8000:8000 ₩
  -v ai_model_cache:/app/model_cache ₩
  -e HF_HOME=/app/model_cache ₩
  -e TRANSFORMERS_CACHE=/app/model_cache ₩
  -e HF_TOKEN=hf_xxxxx ₩
ai-fastapi:latest

# ——— 멀티 스테이지 빌드로 이미지 크기 최소화 ———
# Dockerfile.multistage

# Stage 1: 빌드 환경 (큰 이미지, 임시)
FROM python:3.11 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Stage 2: 실행 환경 (최소 이미지)
FROM python:3.11-slim AS runtime
WORKDIR /app

# builder에서 설치된 패키지만 복사
COPY --from=builder /root/.local /root/.local
COPY . .

ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

# 멀티스테이지 빌드 결과: 최종 이미지 크기 40~60% 감소
# docker build -f Dockerfile.multistage -t ai-fastapi:slim .

```

[코드 8-10] 모델 볼륨 마운트 전략 및 멀티 스테이지 빌드

(4) Docker Compose 소개 및 Gunicorn과 연동 예시

Docker Compose는 여러 컨테이너를 하나의 YAML 파일(docker-compose.yml)로 정의하고 함께 실행하는 도구입니다. FastAPI 앱 + Nginx 리버스 프록시 + 모니터링 도구를 한 번에 구성하는 데 유용합니다.

```
# docker-compose.yml - FastAPI + Nginx + Prometheus 통합 구성
```

```
version: "3.9"
```

```
services:
```

```
  # ——— FastAPI AI 앱 ———
```

```
  api:
```

```
    build:
```

```
      context: .
```

```
      dockerfile: Dockerfile
```

```
    image: ai-fastapi:latest
```

```
    container_name: ai_api
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      - ENVIRONMENT=production
```

```
      - HF_TOKEN=${HF_TOKEN}
```

```
      - MODEL_CACHE_DIR=/app/model_cache
```

```
      - WORKERS=2
```

```
    volumes:
```

```
      - model_cache:/app/model_cache
```

```
      - ./logs:/app/logs
```

```
    expose:
```

```
      - "8000"      # 외부 직접 노출 안 함 (Nginx 경유)
```

```
    deploy:
```

```
      resources:
```

```
        reservations:
```

```
        devices:
```

```
          - driver: nvidia
```

```
            count: 1
```

```
            capabilities: [gpu]
```

```
    healthcheck:
```

```
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
```

```

    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 60s
    networks:
      - ai_network

# ——— Nginx 리버스 프록시 ———
nginx:
  image: nginx:1.25-alpine
  container_name: nginx
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./nginx/ssl:/etc/nginx/ssl:ro
    - ./logs/nginx:/var/log/nginx
  depends_on:
    api:
      condition: service_healthy
  networks:
    - ai_network

# ——— Prometheus 메트릭 수집 ———
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus_data:/prometheus
  command:
    - "--config.file=/etc/prometheus/prometheus.yml"
    - "--storage.tsdb.retention.time=30d"
  networks:

```

```
- ai_network

# — Grafana 시각화 대시보드 —
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD:-admin}
  volumes:
    - grafana_data:/var/lib/grafana
    - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards
  networks:
    - ai_network

volumes:
  model_cache:
    driver: local
  prometheus_data:
  grafana_data:

networks:
  ai_network:
    driver: bridge

# — 실행 명령어 —
# 전체 스택 시작
# docker compose up -d

# 특정 서비스만 재시작
# docker compose restart api

# 로그 확인
# docker compose logs -f api
```

```
# 전체 중지 및 삭제  
# docker compose down -v
```

[코드 8-11] docker-compose.yml – FastAPI + Nginx + Prometheus + Grafana

8.4 AWS, Render 등 배포 실습

(1) AWS EC2 인스턴스에 Docker 이미지 배포

AWS EC2(Elastic Compute Cloud)는 가상 서버를 임대하는 서비스로, AI API 배포에 가장 널리 사용되는 클라우드 플랫폼입니다. GPU 인스턴스(p3, g4dn, g5 패밀리를 선택하면 고성능 AI 추론이 가능합니다.

EC2 인스턴스 타입	GPU	VRAM	vCPU/RAM	시간당 비용 (USD)	
g4dn.xlarge	NVIDIA T4	16GB	4 vCPU / 16GB	~\$0.526	SD v1 (fp16)
g4dn.2xlarge	NVIDIA T4	16GB	8 vCPU / 32GB	~\$0.752	SD v2
g5.xlarge	NVIDIA A10G	24GB	4 vCPU / 16GB	~\$1.006	SDXL
p3.2xlarge	NVIDIA V100	16GB	8 vCPU / 61GB	~\$3.060	대형 텍스트
t3.medium (CPU)	N/A	N/A	2 vCPU / 4GB	~\$0.0416	CLIP, etc

```
# — Step 1: EC2 인스턴스 생성 (AWS CLI) —————
# AWS CLI 설치 및 설정
pip install awscli
aws configure # Access Key, Secret Key, Region 입력

# 보안 그룹 생성
aws ec2 create-security-group --group-name ai-api-sg --description "AI API Security Group"

# 인바운드 규칙 추가
SG_ID="sg-xxxxxxxxxxxxxxxx" # 위에서 생성된 SG ID

aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 22 --cidr 0.0.0.0/0 # SSH
```

```

aws ec2 authorize-security-group-ingress ₩
--group-id $SG_ID --protocol tcp --port 80 --cidr 0.0.0.0/0 # HTTP

aws ec2 authorize-security-group-ingress ₩
--group-id $SG_ID --protocol tcp --port 443 --cidr 0.0.0.0/0 # HTTPS

# EC2 인스턴스 생성 (g4dn.xlarge, Deep Learning AMI)
aws ec2 run-instances ₩
--image-id ami-0c694ab4d92813c5c ₩
--instance-type g4dn.xlarge ₩
--key-name my-keypair ₩
--security-group-ids $SG_ID ₩
--block-device-mappings "[{₩"DeviceName₩":₩"/dev/sda1₩",₩"Ebs₩":{₩"VolumeSize₩":100}}]" ₩
--count 1 ₩
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=ai-api-server}]"

# —— Step 2: 인스턴스 IP 확인 ——
INSTANCE_ID="i-xxxxxxxxxxxxxxxxxx"
PUBLIC_IP=$(aws ec2 describe-instances ₩
--instance-ids $INSTANCE_ID ₩
--query "Reservations[0].Instances[0].PublicIpAddress" ₩
--output text)
echo "퍼블릭 IP: $PUBLIC_IP"

```

[코드 8-12] AWS CLI로 EC2 인스턴스 생성 및 보안 그룹 설정

```

# —— Step 3: SSH 접속 및 서버 초기 설정 ——
ssh -i ~/.ssh/my-keypair.pem ubuntu@$PUBLIC_IP

# —— Step 4: 서버에서 Docker 설치 ——
# (AWS Deep Learning AMI에는 Docker가 이미 설치됨)
# 직접 설치 필요 시:
curl -fsSL https://get.docker.com -o get-docker.sh && sh get-docker.sh
sudo usermod -aG docker ubuntu
newgrp docker

# NVIDIA Container Toolkit 설치 (GPU 컨테이너용)

```

```

distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | \
  sudo tee /etc/apt/sources.list.d/nvidia-docker.list
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit
sudo systemctl restart docker

# —— Step 5: 프로젝트 클론 및 환경 설정 ——
git clone https://github.com/my-username/ai-fastapi.git
cd ai-fastapi

# .env 파일 생성
cat > .env << EOF
ENVIRONMENT=production
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXX
WORKERS=2
LOG_LEVEL=INFO
SECRET_KEY=$(openssl rand -hex 32)
EOF

# —— Step 6: Docker 이미지 빌드 및 실행 ——
docker compose up -d --build

# 서비스 상태 확인
docker compose ps
docker compose logs -f api

# —— Step 7: 헬스체크 ——
curl http://localhost:8000/health
# {"status":"healthy","model_loaded":true,"device":"cuda"}

# 외부에서 접근 테스트
curl http://$PUBLIC_IP/health # Nginx 통해 접근

```

[코드 8-13] EC2 서버 초기 설정 및 Docker Compose 배포

(2) SSH 접속, 포트 설정, 방화벽, Nginx 연동까지 실습

운영 환경에서 FastAPI 서비스는 직접 외부에 노출하지 않고 Nginx를 리버스 프록시로 앞에 두는 것이 표준 아키텍처입니다. Nginx가 HTTPS 종료(TLS Termination), 요청 제한(Rate Limiting), 정적 파일 서빙을 담당하고, FastAPI는 API 로직에만 집중합니다.

```
# /etc/nginx/nginx.conf - Nginx 리버스 프록시 설정
user nginx;
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # — 보안 헤더 —
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin" always;

    # — 요청 크기 제한 —
    client_max_body_size 25M; # 이미지 업로드 허용
    client_body_timeout 120s;

    # — 압축 설정 —
    gzip on;
    gzip_types application/json text/plain;

    # — Rate Limiting —
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
```

```
upstream fastapi_backend {
    server api:8000;
    keepalive 32;
}

# — HTTP → HTTPS 리다이렉트 —
server {
    listen 80;
    server_name myapp.com www.myapp.com;
    return 301 https://$server_name$request_uri;
}

# — HTTPS 서버 —
server {
    listen 443 ssl http2;
    server_name myapp.com www.myapp.com;

    ssl_certificate /etc/nginx/ssl/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
}

# — API 프록시 —
location /api/ {
    limit_req zone=api_limit burst=20 nodelay;
    proxy_pass http://fastapi_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 120s;
    proxy_send_timeout 120s;
    proxy_request_buffering off;
}

# — Swagger UI 접근 —
location /docs {
    proxy_pass http://fastapi_backend;
```

```

    proxy_set_header Host $host;
}

# —— 헬스체크 ——
location /health {
    proxy_pass http://fastapi_backend;
    access_log off;
}
}
}

# —— Let's Encrypt SSL 인증서 발급 (certbot) ——
# sudo apt install certbot python3-certbot-nginx
# sudo certbot --nginx -d myapp.com -d www.myapp.com
# sudo certbot renew --dry-run # 갱신 테스트

```

[코드 8-14] Nginx 리버스 프록시 + HTTPS + Rate Limiting 설정

(3) Render, Railway.io 등 무료/저비용 배포 플랫폼

AWS EC2는 강력하지만 설정이 복잡하고 비용이 높습니다. GPU 없이 CPU만으로 충분한 경량 AI 서비스나 학습/시연 목적이라면 PaaS(Platform as a Service) 플랫폼이 훨씬 간편합니다.

플랫폼	무료 플랜	유료 플랜 시작가	GPU 지원	배포 방법	
Render	512MB RAM , 0.1CPU	\$7/월~	N/A	Git 연동 자동	간편한 V job
Railway.io	\$5 크레딧/월	사용량 기반	N/A	Git/Docker	빠른 배포
Fly.io	256MB RAM , 공 유	\$1.94/월~	N/A	flyctl deploy	옛지 배포
Deta Space	무제한 (경량)	N/A	N/A	space push	서버리스
Heroku	제거됨 (유료만)	\$5/월~	N/A	git push heroku	오래된 배포
Modal.com	\$30 크레딧/월	사용량 기반	A10G , A100	modal deploy	AI 전용,

```
# —— Render 배포 방법 ——
```

```
# 1. render.com 접속 → New → Web Service
# 2. GitHub 저장소 연결
# 3. Build Command: pip install -r requirements.txt
# 4. Start Command: uvicorn main:app --host 0.0.0.0 --port $PORT
# 5. 환경변수 설정 (HF_TOKEN 등)
```

```
# render.yaml (인프라 코드로 Render 설정)
```

```
services:
```

```
- type: web
  name: ai-fastapi
  env: python
  buildCommand: "pip install -r requirements.txt"
  startCommand: "uvicorn main:app --host 0.0.0.0 --port $PORT"
  envVars:
    - key: HF_TOKEN
      sync: false # 수동으로 Render 대시보드에서 설정
    - key: ENVIRONMENT
      value: production
  healthCheckPath: /health
  autoDeploy: true
```

```
# ——— Railway.io 배포 방법 ———
```

```
# pip install railway
# railway login
# railway init # 새 프로젝트 생성
# railway up # 배포
```

```
# Procfile (Railway/Heroku용)
```

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

```
# ——— Modal.com GPU 서버리스 배포 ———
```

```
# pip install modal
# modal token new # 토큰 발급
```

```
import modal
```

```
app = modal.App("ai-fastapi")

image = modal.Image.debian_slim().pip_install(
    "fastapi", "uvicorn", "transformers", "torch",
    "diffusers", "accelerate"
)

@app.function(
    gpu="A10G",
    image=image,
    timeout=300,
    container_idle_timeout=60
)
@modal.asgi_app()
def fastapi_app():
    from main import app as fastapi_app
    return fastapi_app

# 배포: modal deploy modal_app.py
# 테스트: modal run modal_app.py
```

[코드 8-15] Render, Railway, Modal.com 배포 설정 예시

(4)(5) Hugging Face 토큰/모델 캐싱 이슈 해결 및 GitHub 자동 배포

클라우드 배포 시 가장 자주 발생하는 문제들과 GitHub Actions를 이용한 CI/CD 자동 배포 파이프라인 구성 방법을 다룹니다.

🔑 배포 시 자주 발생하는 문제와 해결 방법

문제 1: "Repository not found" - HF 토큰 미설정

해결: 환경변수 HUGGING_FACE_HUB_TOKEN 또는 HF_TOKEN 설정

`export HF_TOKEN=hf_xxxx` 또는 `.env` 파일에 추가

문제 2: 컨테이너 재시작마다 모델 재다운로드

해결: Docker 볼륨으로 HF_HOME 캐시 디렉토리를 영구 마운트

`-v model_volume:/root/.cache/huggingface`

문제 3: "No space left on device" - 모델 파일이 디스크를 가득 채움

해결: EC2 볼륨 크기 증가 (`aws ec2 modify-volume`) 또는

`/tmp` 대신 별도 EBS 볼륨을 `/data`에 마운트

문제 4: 첫 요청이 매우 느림 (Warm-up 문제)

해결: 앱 시작 시 워밍업 추론 실행 (lifespan에서 더미 추론)

또는 Gunicorn `worker_init` 훅에서 모델 사전 로드

문제 5: GPU OOM (Out of Memory)

해결: 워커 수 감소, fp16/int4 양자화 적용, attention_slicing 활성화

.github/workflows/deploy.yml - GitHub Actions CI/CD 파이프라인

name: AI API Deploy to AWS EC2

on:

push:

branches: [main] # main 브랜치 푸시 시 자동 배포

```

workflow_dispatch: # 수동 트리거 허용

env:
  DOCKER_IMAGE: ${ secrets.DOCKERHUB_USERNAME }}/ai-fastapi
  EC2_HOST: ${ secrets.EC2_PUBLIC_IP }

jobs:
  # — Step 1: 코드 테스트 —
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest httpx

      - name: Run tests
        run: pytest tests/ -v --tb=short

  # — Step 2: Docker 이미지 빌드 및 푸시 —
  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:

```

```

    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: |
      ${ env.DOCKER_IMAGE }:latest
      ${ env.DOCKER_IMAGE }:${ github.sha }
    cache-from: type=gha
    cache-to: type=gha,mode=max

# — Step 3: EC2 서버 배포 —————
deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  steps:
    - name: Deploy to EC2
      uses: appleboy/ssh-action@v1
      with:
        host: ${ env.EC2_HOST }
        username: ubuntu
        key: ${ secrets.EC2_SSH_PRIVATE_KEY }
        script: |
          cd /home/ubuntu/ai-fastapi
          docker compose pull api
          docker compose up -d --no-deps api
          docker system prune -f --filter "until=24h"
          echo "배포 완료: $(date)"

```

[코드 8-16] GitHub Actions 기반 CI/CD 자동 배포 파이프라인

8.5 API 모니터링과 속도 최적화

(1) FastAPI에서 요청/응답 로그 남기기 (LoggingMiddleware)

운영 중인 API에서 발생하는 요청과 응답을 체계적으로 로깅하는 것은 장애 추적, 성능 분석, 보안 감사에 필수적입니다. FastAPI의 미들웨어(Middleware) 기능을 활용하면 모든 요청에 대해 일관된 로그를 남길 수 있습니다.

```
# logging_middleware.py - 요청/응답 로깅 미들웨어
import time, uuid, json
from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware
from loguru import logger

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    """모든 HTTP 요청/응답을 구조화된 JSON으로 로깅합니다."""

    # 로깅 제외 경로 (헬스체크, 메트릭 등)
    SKIP_PATHS = {"/health", "/metrics", "/favicon.ico"}

    async def dispatch(self, request: Request, call_next) -> Response:
        # 헬스체크 경로 스킵
        if request.url.path in self.SKIP_PATHS:
            return await call_next(request)

        # 요청 ID 생성 (추적용)
        request_id = str(uuid.uuid4())[:8]
        request.state.request_id = request_id

        # 요청 정보 수집
        start_time = time.time()

        client_ip = request.headers.get("X-Real-IP") or ₩
```

```

        request.headers.get("X-Forwarded-For", "").split(",")[0].strip() or \
        (request.client.host if request.client else "unknown")

log_data = {
    "request_id": request_id,
    "method": request.method,
    "path": request.url.path,
    "query": str(request.url.query) or None,
    "client_ip": client_ip,
    "user_agent": request.headers.get("User-Agent", "")[:100],
}

# 요청 처리
try:
    response = await call_next(request)
    status_code = response.status_code
except Exception as exc:
    elapsed_ms = (time.time() - start_time) * 1000
    logger.error({
        **log_data,
        "status": 500,
        "elapsed_ms": round(elapsed_ms, 2),
        "error": str(exc)
    })
    raise

# 응답 로깅
elapsed_ms = (time.time() - start_time) * 1000
log_entry = {
    **log_data,
    "status": status_code,
    "elapsed_ms": round(elapsed_ms, 2),
}

# 상태 코드에 따라 로그 레벨 분류
if status_code >= 500:
    logger.error(json.dumps(log_entry, ensure_ascii=False))
elif status_code >= 400:

```

```

        logger.warning(json.dumps(log_entry, ensure_ascii=False))
    elif elapsed_ms > 5000:
        logger.warning(json.dumps(**log_entry, "slow_request": True, ensure_ascii=False))
    else:
        logger.info(json.dumps(log_entry, ensure_ascii=False))

# 응답 헤더에 요청 ID 추가
response.headers["X-Request-ID"] = request_id
response.headers["X-Response-Time"] = f"{elapsed_ms:.2f}ms"
return response

# ——— loguru 설정 (main.py에서 호출) ———
from loguru import logger
import sys

def setup_logging(log_file: str = "./logs/app.log"):
    """loguru 핸들러 설정"""
    import os; os.makedirs(os.path.dirname(log_file), exist_ok=True)

    # 기본 핸들러 제거
    logger.remove()

    # 콘솔 출력 (색상 포함)
    logger.add(sys.stderr, level="DEBUG" if os.getenv("DEBUG") else "INFO",
               format="<green>{time:HH:mm:ss}</green> | <level>{level:7}</level> | {message}")

    # 파일 로테이션 (하루 단위, 최대 30일 보관)
    logger.add(log_file,
               rotation="00:00",    # 자정마다 새 파일
               retention="30 days",  # 30일 후 자동 삭제
               compression="gz",    # 압축 보관
               level="INFO",
               encoding="utf-8",
               serialize=True)      # JSON 형식으로 저장

# main.py 에서 사용

```

```
# from logging_middleware import RequestLoggingMiddleware
# from logging_setup import setup_logging
# setup_logging()
# app.add_middleware(RequestLoggingMiddleware)
```

[코드 8-17] 요청/응답 로깅 미들웨어 + loguru 설정

(2) Uvicorn/Gunicorn 로그 설정

Uvicorn과 Gunicorn의 로그 설정을 통해 서버 레벨의 액세스 로그와 에러 로그를 체계적으로 관리합니다. JSON 형식의 구조화 로그는 Elasticsearch, Loki 등 로그 수집 시스템과 쉽게 통합됩니다.

```
# uvicorn_logging.py – Uvicorn JSON 구조화 로그 설정
import logging
import json
from datetime import datetime

class UvicornJSONFormatter(logging.Formatter):
    """Uvicorn 로그를 JSON 형식으로 출력하는 포맷터"""

    def format(self, record: logging.LogRecord) -> str:
        log_data = {
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }
        if record.exc_info:
            log_data["exception"] = self.formatException(record.exc_info)
        return json.dumps(log_data, ensure_ascii=False)

# Uvicorn 로깅 설정 디렉터리 (main.py에서 사용)
LOGGING_CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "json": {
            "()": "uvicorn_logging.UvicornJSONFormatter",
        },
    },
    "handlers": {
```

```

    "console": {
        "class": "logging.StreamHandler",
        "formatter": "json",
        "stream": "ext://sys.stdout"
    },
    "file": {
        "class": "logging.handlers.TimedRotatingFileHandler",
        "formatter": "json",
        "filename": "./logs/uvicorn.log",
        "when": "midnight",
        "backupCount": 30,
        "encoding": "utf-8"
    }
},
"loggers": {
    "uvicorn": {"level": "INFO", "handlers": ["console", "file"]},
    "uvicorn.error": {"level": "INFO", "handlers": ["console", "file"]},
    "uvicorn.access": {"level": "INFO", "handlers": ["file"]},
    "fastapi": {"level": "INFO", "handlers": ["console", "file"]},
}
}

```

Uvicorn 실행 시 로깅 설정 적용

uvicorn main:app --log-config uvicorn_logging.py

— 코드에서 직접 적용 —

```

import uvicorn
from config import get_settings

if __name__ == "__main__":
    settings = get_settings()
    uvicorn.run(
        "main:app",
        host=settings.HOST,
        port=settings.PORT,
        workers=settings.WORKERS,
        log_config=LOGGING_CONFIG,
        reload=not settings.is_production,
    )

```

```
access_log=True
)
```

[코드 8-18] Uvicorn JSON 구조화 로그 설정

(3)(4) 응답 속도 측정 및 병목 원인 분석과 개선 전략

AI API의 응답 속도는 사용자 경험에 직결됩니다. 느린 응답의 원인을 정확히 파악하고 단계별로 최적화하는 체계적인 접근이 필요합니다.

```
# performance_profiler.py - 엔드포인트 성능 측정 도구
import time, asyncio
import functools
from fastapi import Request
from loguru import logger

# ——— 데코레이터 기반 함수 수준 프로파일링 ———
def profile_async(func):
    """비동기 함수의 실행 시간을 측정하는 데코레이터"""
    @functools.wraps(func)
    async def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = await func(*args, **kwargs)
        elapsed = (time.perf_counter() - start) * 1000
        logger.debug(f"[PROFILE] {func.__name__}: {elapsed:.2f}ms")
        if elapsed > 1000:
            logger.warning(f"[SLOW] {func.__name__}: {elapsed:.2f}ms 초과")
        return result
    return wrapper

# ——— 단계별 시간 측정 컨텍스트 매니저 ———
class Timer:
    """코드 블록의 실행 시간을 측정합니다."""
```

```

def __init__(self, label: str = ""):
    self.label = label
    self.checkpoints = {}
    self._start = None

def start(self):
    self._start = time.perf_counter()
    return self

def checkpoint(self, name: str):
    """중간 지점 시간 기록"""
    self.checkpoints[name] = (time.perf_counter() - self._start) * 1000

def stop(self) -> dict:
    total = (time.perf_counter() - self._start) * 1000
    return {
        "label": self.label,
        "total_ms": round(total, 2),
        "checkpoints": {k: round(v, 2) for k, v in self.checkpoints.items()}
    }

# ——— 실제 사용 예시 ———
@app.post("/api/v1/generate")
async def generate_image(request: ImageGenRequest):
    timer = Timer("image_generation").start()

    # 단계 1: 입력 검증
    validated = validate_request(request)
    timer.checkpoint("validation")

    # 단계 2: 전처리
    processed = preprocess(validated)
    timer.checkpoint("preprocessing")

    # 단계 3: 모델 추론 (가장 오래 걸리는 단계)
    result = await run_inference(processed)

```

```

timer.checkpoint("inference")

# 단계 4: 후처리
response = postprocess(result)
timer.checkpoint("postprocessing")

timing = timer.stop()
logger.info(f"타이밍: {timing}")
# 출력 예: {"label": "image_generation", "total_ms": 3420.15,
# "checkpoints": {"validation": 2.1, "preprocessing": 45.3,
# "inference": 3350.2, "postprocessing": 22.5}}

return response

```

[코드 8-19] 단계별 성능 프로파일링 도구

병목 유형	증상	원인	해결 방법
모델 로딩 지연	첫 요청만 30~60초 느림	모델 지연 로드(lazy load)	lifespan에서 미리 로드 + 워밍업 추론
GPU 메모리 부 족	간헐적 OOM 에러	동시 요청이 VRAM 초과	inference_lock으로 직렬화, 워커 수 감소
CPU 병목	모든 요청이 균일하게 느림	fp32, 불필요한 텐서 변환	fp16 전환, torch.inference_mode()
디스크 I/O	이미지 저장/읽기 느 림	동기적 파일 I/O	aiofiles로 비동기 처리, 메모리 처 리
네트워크 지연	빠른 추론인데 응답 느림	큰 이미지 직렬화, 압 축 없음	JPEG 압축, base64 캐싱, CDN
프리웨이팅	동시 요청 시 큐잉	단일 워커, 직렬 처리	Gunicorn 멀티워커, 비동기 처리

(5) 실시간 모니터링 도구: Prometheus, Grafana, Sentry

프로덕션 AI 서비스를 안정적으로 운영하기 위해서는 실시간 메트릭 수집(Prometheus), 시각화 대시보드(Grafana), 오류 추적(Sentry)의 세 가지 도구가 핵심입니다.

```
# monitoring.py - Prometheus 메트릭 설정
# pip install prometheus-fastapi-instrumentator

from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator, metrics
from prometheus_client import Counter, Histogram, Gauge
import torch

# ——— 커스텀 메트릭 정의 ———
MODEL_INFERENCE_TIME = Histogram(
    "ai_model_inference_seconds",
    "AI 모델 추론 시간 (초)",
    buckets=[0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 30.0, 60.0]
)

MODEL_REQUESTS_TOTAL = Counter(
    "ai_model_requests_total",
    "AI 모델 총 요청 수",
    labelnames=["model_name", "status"]
)

GPU_MEMORY_USED = Gauge(
    "gpu_memory_used_bytes",
    "GPU 메모리 사용량 (bytes)",
    labelnames=["device"]
)

ACTIVE_REQUESTS = Gauge(
    "ai_active_requests",
    "현재 처리 중인 요청 수"
```

```

)

def setup_prometheus(app: FastAPI) -> None:
    """FastAPI 앱에 Prometheus 계측기를 부착합니다."""

    instrumentator = Instrumentator(
        should_group_status_codes=False,
        should_respect_env_var=True,
        env_var_name="ENABLE_METRICS",
        excluded_handlers=["/health", "/metrics"],
    )

    # 기본 HTTP 메트릭 추가
    instrumentator.add(metrics.request_size())
    instrumentator.add(metrics.response_size())
    instrumentator.add(metrics.latency(buckets=[0.1, 0.5, 1, 5, 10, 30]))

    # /metrics 엔드포인트 등록 및 계측 시작
    instrumentator.instrument(app).expose(app, endpoint="/metrics")

# — GPU 메트릭 주기적 업데이트 —
async def update_gpu_metrics():
    """GPU 메모리 사용량을 주기적으로 Prometheus에 기록합니다."""
    while True:
        if torch.cuda.is_available():
            for i in range(torch.cuda.device_count()):
                used = torch.cuda.memory_allocated(i)
                GPU_MEMORY_USED.labels(device=f"cuda:{i}").set(used)
            await asyncio.sleep(15) # 15초마다 갱신

# — Sentry 에러 추적 —
# pip install sentry-sdk[fastapi]
import sentry_sdk

```

```
from sentry_sdk.integrations.fastapi import FastApiIntegration
from sentry_sdk.integrations.sqlalchemy import SQLAlchemyIntegration
```

```
def setup_sentry(dsn: str, environment: str) -> None:
    """Sentry 에러 추적 초기화"""
    sentry_sdk.init(
        dsn=dsn,
        integrations=[
            FastApiIntegration(transaction_style="endpoint"),
        ],
        environment=environment,
        traces_sample_rate=0.1,    # 10% 트랜잭션 추적
        profiles_sample_rate=0.1,  # 10% 성능 프로파일링
        send_default_pii=False,    # 개인정보 전송 금지
    )
```

```
# — Prometheus 설정 파일 —
```

```
# monitoring/prometheus.yml
```

```
global:
```

```
  scrape_interval: 15s
```

```
  evaluation_interval: 15s
```

```
scrape_configs:
```

```
- job_name: "ai-fastapi"
```

```
  static_configs:
```

```
    - targets: ["api:8000"]
```

```
  metrics_path: /metrics
```

```
- job_name: "node-exporter"
```

```
  static_configs:
```

```
    - targets: ["node-exporter:9100"]
```

[코드 8-20] Prometheus 메트릭 + Sentry 에러 추적 통합 설정

```
# — 엔드포인트에서 메트릭 기록 예시 —
```

```
from monitoring import MODEL_INFERENCE_TIME, MODEL_REQUESTS_TOTAL, ACTIVE_REQUESTS
```

```

@app.post("/api/v1/generate")
async def generate_with_metrics(request: ImageGenRequest):
    ACTIVE_REQUESTS.inc()
    start = time.time()
    try:
        result = await do_inference(request)
        MODEL_REQUESTS_TOTAL.labels(
            model_name="stable-diffusion", status="success"
        ).inc()
        return result
    except Exception as e:
        MODEL_REQUESTS_TOTAL.labels(
            model_name="stable-diffusion", status="error"
        ).inc()
        # Sentry로 에러 자동 전송
        raise
    finally:
        elapsed = time.time() - start
        MODEL_INFERENCE_TIME.observe(elapsed)
        ACTIVE_REQUESTS.dec()

# — Grafana 대시보드 쿼리 예시 (PromQL) —
# 초당 요청 수 (RPS)
# rate(http_requests_total{job="ai-fastapi"}[5m])

# 평균 응답 시간
# rate(http_request_duration_seconds_sum[5m])
# / rate(http_request_duration_seconds_count[5m])

# P95 응답 시간
# histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# AI 추론 P99 지연 시간
# histogram_quantile(0.99, rate(ai_model_inference_seconds_bucket[5m]))

# 에러율

```

```
# rate(http_requests_total{status=~"5.."}[5m])
# / rate(http_requests_total[5m]) * 100

# GPU 메모리 사용률 (%)
# gpu_memory_used_bytes / gpu_memory_total_bytes * 100
```

[코드 8-21] 메트릭 기록 엔드포인트 및 Grafana PromQL 쿼리 예시

모니터링 도구	목적	핵심 기능	설치 난이도
Prometheus	메트릭 수집/저장	시계열 DB, 알람 규칙 (AlertManager)	중간 (설정 파일 작성)
Grafana	메트릭 시각화	대시보드, 알람, 다양한 데이터소스	쉬움 (웹 UI 설정)
Sentry	에러 추적/알림	스택 트레이스, 이슈 그룹화, Slack 연동	쉬움 (SDK 한 줄)
Flower (Celery)	비동기 작업 모니터링	태스크 큐 상태, 워커 현황	중간 (Celery 필요)
Loki + Promtail	로그 수집/검색	Grafana와 통합, 로그 쿼리	중간 (에이전트 설치)

8장 마무리

이 장에서는 FastAPI 기반 AI 서비스를 운영 환경에 배포하고 안정적으로 유지하는 전 과정을 실습했습니다. Uvicorn+Gunicorn의 운영 서버 구성부터 Docker 컨테이너화, AWS/Render 배포, Nginx 리버스 프록시, GitHub Actions CI/CD, 그리고 Prometheus+Grafana+Sentry 모니터링 스택까지, 실제 프로덕션 환경에서 필요한 모든 핵심 요소를 다루었습니다.

섹션	핵심 기술	실무 적용
8.1 운영 서버	Gunicorn + UvicornWorker, gunicorn.conf.py	멀티 워커, 자동 재시작, 설정 파일 관리
8.2 HF Spaces	Gradio/Streamlit 앱, app.py, README.md	학습/데모용 무료 배포, 빠른 프로토타이핑
8.3 Docker	Dockerfile, .dockerignore, Compose	환경 일관성, 이식성, 재현 가능 배포
8.4 클라우드 배포	AWS EC2, Nginx, Render, GitHub Actions	CI/CD 자동화, 무중단 배포, 인프라 코드화
8.5 모니터링	Prometheus, Grafana, Sentry, loguru	실시간 성능 추적, 장애 조기 발견, 최적화

🔗 다음 단계 학습 로드맵

이 교재를 마친 후 다음 주제들로 학습을 이어가세요:

- Kubernetes(K8s): Docker Compose를 넘어 대규모 컨테이너 오케스트레이션
- MLOps: MLflow, DVC로 모델 버전 관리 및 실험 추적
- Ray Serve: 분산 AI 모델 서빙 프레임워크
- vLLM: 대형 언어 모델 고처리량 서빙 엔진
- Terraform: AWS/GCP 인프라를 코드로 관리 (IaC)

연습 문제

1. 8.1절의 `gunicorn.conf.py`를 로컬 환경에서 작성하고, `workers=4`로 설정하여 Gunicorn을 실행해 보세요. 그런 다음 `wrk` 또는 `locust` 부하 테스트 도구로 동시 요청 10개를 보내 응답 시간 분포를 측정하고, `workers=1`로 변경했을 때의 차이를 비교하세요.
2. Hugging Face Spaces에 무료 계정으로 Gradio Space를 생성하고, 7장에서 만든 멀티모달 챗봇 앱을 `app.py`로 작성하여 배포해 보세요. 배포된 Space URL에서 이미지를 업로드하여 동작을 확인하고, Space의 Settings에서 하드웨어를 CPU Basic과 T4 Small(GPU)으로 각각 설정하여 응답 속도를 비교하세요.
3. 이 장의 Dockerfile을 사용하여 4~7장에서 개발한 FastAPI AI API를 컨테이너 이미지로 빌드하세요. 이미지를 성공적으로 빌드한 후 `docker run`으로 실행하고, `curl`로 `/health` 엔드포인트를 호출하여 정상 동작을 확인하세요. 최종 이미지 크기를 `docker images` 명령으로 확인하고, `.dockerignore`를 최적화하여 크기를 줄여보세요.
4. 8.5절의 `RequestLoggingMiddleware`와 Prometheus 설정을 FastAPI 앱에 적용하고, `/metrics` 엔드포인트에서 수집되는 메트릭을 확인하세요. 그런 다음 `docker compose up`으로 Prometheus와 Grafana를 함께 실행하고, Grafana 대시보드에서 API 요청 수와 평균 응답 시간을 시각화하는 패널 2개를 생성하세요.
5. (심화) GitHub Actions CI/CD 파이프라인(`deploy.yml`)을 작성하여, main 브랜치에 코드를 푸시하면 자동으로 (A) pytest 테스트 실행, (B) Docker 이미지 빌드 및 Docker Hub 푸시, (C) Render 또는 Fly.io에 자동 배포가 이루어지는 전체 파이프라인을 구성하세요. 각 스텝이 성공/실패했을 때 GitHub Actions 화면에서 결과를 확인하는 방법도 기술하세요.

— 8장 끝 / 교재 전체 완결 —